

人工智能：  
不属于人工智能：自动控制，计算科学  
图灵机的定义：是一个数学模型，是一个七元组  
人工智能三大流派：符号主义，连接主义，行为主义  
符号主义：把现实的物，映射到代表它的符号，在符号上完成所有的推理、计算等等。图灵测试就是这个思路

超算主要是浮点运算能力很强，普通计算机计算浮点运算很慢  
GPU 无法单独工作，必须由 CPU 进行控制调用才能工作。CPU 可单独作用，处理复杂的逻辑运算和不同的数据类型，但当需要大量的处理类型统一的数据时，则可调用 GPU 进行并行计算。  
“天河一号”第一次采用 GPU 承担主要计算量的异构模式

大数据（big data），指无法在一定时间范围内用常规软件工具进行捕捉、管理和处理的数据集合，是需要新处理模式才能具有更强的决策力、洞察发现力和流程优化能力的海量、高增长率和多样化的信息资产。  
大数据的 5V 特点（IBM 提出）：Volume（大量）、Velocity（高速）、Variety（多样）、Value（低价值密度）、Veracity（真实性）。

| 求解过程                                                                                                                                                                                                                                  |         |       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|-------|
| 数据库                                                                                                                                                                                                                                   | 可触发规则   | 被触发规则 |
| A, B                                                                                                                                                                                                                                  | (1)     | (1)   |
| A, B, C                                                                                                                                                                                                                               | (2) (3) | (2)   |
| A, B, C, D                                                                                                                                                                                                                            | (3) (5) | (3)   |
| A, B, C, D, G                                                                                                                                                                                                                         | (5)     | (5)   |
| A, B, C, D, G, E                                                                                                                                                                                                                      | (4)     | (4)   |
| A, B, C, D, G, E, F                                                                                                                                                                                                                   |         |       |
| <div>1, IF A<math>\wedge</math>B THEN C                      2, IF A<math>\wedge</math>C THEN D</div> <div>3, IF B<math>\wedge</math>C THEN G                      4, IF B<math>\wedge</math>E THEN F</div> <div>5, IF D THEN E</div> |         |       |

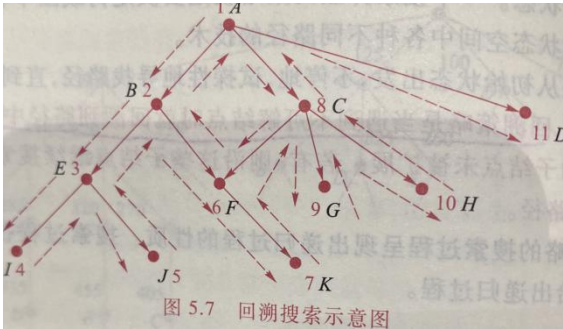
有点像规约，第一个 A 和 B then C，然后把 C 入栈，然后把和 C 关联 (2) & (3) 的入栈，然后二者随便选一个，选了 (2) 可以规约出 D 的把 D 入栈。

# M-C问题（续3）

## 4，规则集

- IF (m, c, 1) THEN (m-1, c, 0)
- IF (m, c, 1) THEN (m, c-1, 0)
- IF (m, c, 1) THEN (m-1, c-1, 0)
- IF (m, c, 1) THEN (m-2, c, 0)
- IF (m, c, 1) THEN (m, c-2, 0)

产生式特点：  
数据驱动，知识的无序性，控制系统与问题无关，数据、知识和控制相互独立。  
盲目算法：回溯，宽度优先，深度优先  
回溯算法：遇到不行的解就掉头。深度优先，但可能会死循环，可以限定深度或者记录路径  
回溯搜索：只保存从当前路径到下一节点的值；  
PS（访问过的点） NPS（将要访问的点） NSS（不可访问的点） cs 是正在访问的点  
Ps = [start] Nps = [start]  
NSS = [0]



值:PS=[A]; NPS=[A]; NSS=[ ]; CS=A

| 重复 | CS | PS     | NPS         | NSS      |
|----|----|--------|-------------|----------|
| 0  | A  | [A]    | [A]         | [ ]      |
| 1  | B  | [BA]   | [BCDA]      | [ ]      |
| 2  | E  | [EBA]  | [EFBDCA]    | [ ]      |
| 3  | I  | [IEBA] | [IJEFBBCDA] | [ ]      |
| 4  | J  | [JEBA] | [JEFBCDA]   | [I]      |
| 5  | F  | [FBA]  | [FBCDA]     | [EJI]    |
| 6  | K  | [KFBA] | [KFBCDA]    | [EJI]    |
| 7  | C  | [CA]   | [CDA]       | [BFKEJI] |
| 8  | G  | [GCA]  | [GHCDA]     | [BFKEJI] |

下一个必考：

**图搜索：**保留所有路径

八数码是按照空格移动的方向来生成树的，不一定是第一个根生成四个节点的，若是边上的空格可能会只有三个或者两个方向。

**宽度优先：**

Open 表相当于 NPS 表，表示未被搜索的状态

Close 表相当于 PS 和 NSS 的合并表，记录了已经生成并扩展的状态

While open! = 空

    从 open 删除第一个值

    放入 close 中

    如果等于目标状态的话就 return succeed

    Else 生成这个节点所有的子状态

    删除已经在 open 和 close 的节点，然后剩下的加入 open

End

Open 是先进先出的队列

但是宽度优先搜索太费空间，因为要把一层都加入 OPEN 表

但是总能找到最优的解

**深度优先：**

While open! = 空

    从 open 删除第一个值

    放入 close 中

    如果等于目标状态的话就 return succeed

    If 节点的深度 < d 就继续；

    Else 生成这个节点所有的子状态

    删除已经在 open 和 close 的节点，然后剩下的加入 open

End

**深度优先的 open 表是栈的结构先进后出**

并且不一定可以找到最优的解

启发式搜索：利用知识来引导搜索，达到减少搜索范围，降低问题复杂度的目的。

启发式搜索的强度：

强：降低搜索工作量，但可能导致找不到最优解

弱：一般导致工作量加大，极端情况下变为盲目搜索，但可能可以找到最优解

启发式搜索：

可以利用启发式搜索的剪枝来减少空间的大小

可以利用搜索消除复杂的搜索过程

启发式搜索的方法的种类：陈述性（描述更精炼具体），过程性（操作算子少而精），控制性（控制策略）

估价函数（从初始点经过 n 节点到终点的路径最小的代价估计值）：

$F(n) = g(n) + h(n)$

G(n)是实际从初始点到 n 节点的代价

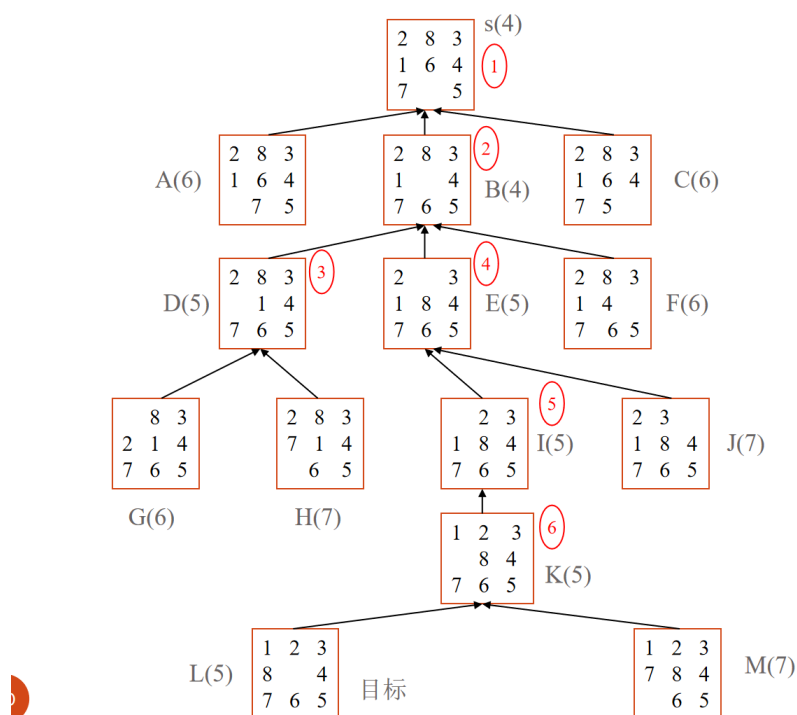
H(n)是从 n 到终点的估计代价

只希望找到目的节点而不关心代价，就忽略 g(n)

当  $h(n) \gg g(n)$  的时候，有利于提高效率的搜索，但影响效率的完备性

A 搜索算法：

$G(n)$  代表状态的深度， $h(n)$  代表不在位数码的个数。



最后 close 的存储的节点是解的路径

删除 open 的第一个节点并加到 close 中，把他的子节点的状态按照  $f(n)$  的排序加进来

表 5.1 搜索过程中 open 表和 closed 表内状态排列的变化情况

| open 表                            | closed 表           |
|-----------------------------------|--------------------|
| 初始化: $(s(4))$                     | $()$               |
| 一次循环后:                            |                    |
| $(B(4) A(6) C(6))$                | $(s(4))$           |
| 二次循环后:                            |                    |
| $(D(5) E(5) A(6) C(6) F(6))$      | $(s(4) B(4))$      |
| 三次循环后:                            |                    |
| $(E(5) A(6) C(6) F(6) G(6) H(7))$ | $(s(4) B(4) D(5))$ |
| 四次循环后:                            |                    |

A\* 算法：

在 A 算法中，如果满足条件：

$h^*(n)$  是状态  $n$  到目的状态的最佳路径的代价

$$h(n) \leq h^*(n)$$

则 A 算法称为 A\* 算法。

A\* 算法比 A 算法好，因为 A\* 算法一定能找到最优的解，并且一定是只要问题有解

A\* 算法的特性：

可采纳性，单调性，信息性

可采纳性:

若存在从初始节点  $s$  到目标节点  $t$  有路径, 则  $A^*$  必能找到最佳解结束。

信息性:

如果  $h_2(n) > h_1(n)$  (目标节点除外), 则  $A_1$  扩展的节点数  $\geq A_2$  扩展的节点数  
 $h_2$  的信息比  $h_1$  的多

证明: 用归纳的反证法

- 所以有:  $f_1(n) \geq f^*(s)$
- 即:  $g_1(n) + h_1(n) \geq f^*(s)$
- 所以:  $h_1(n) \geq f^*(s) - g_1(n)$
- 另一方面, 由于  $A_2$  扩展了  $n$ , 有  $f_2(n) \leq f^*(s)$
- 即:  $h_2(n) \leq f^*(s) - g_2(n)$  (A)
- 由于  $d(n)=k$  时,  $A_2$  扩展的节点  $A_1$  一定扩展, 有  
 $g_1(n) \leq g_2(n)$  (因为  $A_2$  的路  $A_1$  均走到了)
- 所以:  $h_1(n) \geq f^*(s) - g_1(n) \geq f^*(s) - g_2(n)$  (B)
- 比较 A、B 两式, 有  $h_1(n) \geq h_2(n)$ , 与定理条件矛盾。故定理得证。

推理:

$A^*$  选作扩展的任一节点  $n$ , 有  $f(n) \leq f^*(s)$

单调性:

启发策略无论是何处都是可采纳的, 总是沿着祖先沿着最佳的路线到达另一状态。当  $A^*$  选  $n$  扩展时, 有  $g(n)=g^*(n)$ 。

若  $h(n)$  是单调的, 则由  $A^*$  所扩展的节点序列其  $f$  值是非递减的。即  $f(n_i) \leq f(n_j)$ 。

证明:

由单调限制条件, 对  $P$  中任意节点  $n_i$  有:

$$h(n_i) \leq C(n_i, n_{i+1}) + h(n_{i+1})$$

$$g^*(n_i) + h(n_i) \leq g^*(n_i) + C(n_i, n_{i+1}) + h(n_{i+1})$$

由于  $n_i$ 、 $n_{i+1}$  在最佳路径上, 所以:

$$g^*(n_{i+1}) = g^*(n_i) + C(n_i, n_{i+1})$$

带入上式有:

$$g^*(n_i) + h(n_i) \leq g^*(n_{i+1}) + h(n_{i+1})$$

从  $i=j$  到  $i=k-1$  应用上不等式, 有:

$$g^*(n_{j+1}) + h(n_{j+1}) \leq g^*(n_k) + h(n_k)$$

$$\text{即: } f(n_{j+1}) \leq g^*(n) + h(n)$$

注意: ( $n_i$  在 CLOSED 中,  $n_{i+1}$  在 OPEN 中)

重写上式:  $f(n_{j+1}) \leq g^*(n) + h(n)$

另一方面,  $A^*$ 选 $n$ 扩展, 必有:

$$f(n) = g(n) + h(n) \leq f(n_{j+1})$$

比较两式, 有:

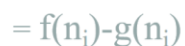
$$g(n) \leq g^*(n)$$

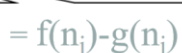
但已知 $g^*(n)$ 是最佳路径的耗散值, 所以只有:  $g(n) = g^*(n)$ 。得证。

第二行的证明:

由单调限制条件, 有:

$$h(n_i) - h(n_j) \leq C(n_i, n_j)$$


$$= f(n_i) - g(n_i)$$


$$= f(n_j) - g(n_j)$$

$$f(n_i) - g(n_i) - f(n_j) + g(n_j) \leq C(n_i, n_j)$$


$$= g(n_i) + C(n_i, n_j)$$

$$f(n_i) - g(n_i) - f(n_j) + g(n_i) + C(n_i, n_j) \leq C(n_i, n_j)$$

$$f(n_i) - f(n_j) \leq 0, \text{ 得证。}$$

一个证明  $h$  效果的式子

- 平均分叉树

设共扩展了 $d$ 层节点, 共搜索了 $N$ 个节点, 则:

其中,  $b^*$ 称为平均分叉树。

$$N = \left(1 - b^{*(d+1)}\right) / (1 - b^*)$$

- $b^*$ 越小, 说明 $h$ 效果越好。
- 实验表明,  $b^*$ 是一个比较稳定的常数, 同一问题基本不随问题规模变化。

- 一般来说，A\*的算法复杂性是指数型的，可以证明，当且仅当以下条件成立时：

$$\text{abs}(h(n)-h^*(n)) \leq O(\log(h^*(n)))$$

A\*的算法复杂性才是非指数型的，但是通常情况下， $h$ 与 $h^*$ 的差别至少是和离目标的距离成正比的。

A 算法与 A\*算法的比较：

因 A 算法第 6 步对 ml 类节点可能要重新放回到 OPEN 表中，因此可能会导致多次重复扩展同一个节点，导致搜索效率下降。

但是，单调性可以对这个加以限制，即让每一步都是最优的即：当 A\*选  $n$  扩展时，有  $g(n)=g^*(n)$ 。

中 $n_j$ 是 $n_i$ 的子节点，满足

$$\begin{cases} h(n_i) - h(n_j) \leq \text{cost}(n_i, n_j) \\ h(t) = 0 \end{cases}$$

或

#### 博弈问题：

博弈是有限的。即无论两人怎样决策，都会在有限步后决出胜负。

巴什博弈：一共  $n$  个，每个人可以拿  $m$  个

那么会有下面的结果：

由此易得：如果  $n\%(m+1) \neq 0$  也就相当于是  $n = (m+1) * r + s$

这种情况下 按照上述所说的方法则先手获胜。

反之如果  $n\%(m+1) = 0$  就相当于是  $n = (m+1) * r$

这种情况后手按照上述方法 后手就能获胜。

极小极大：

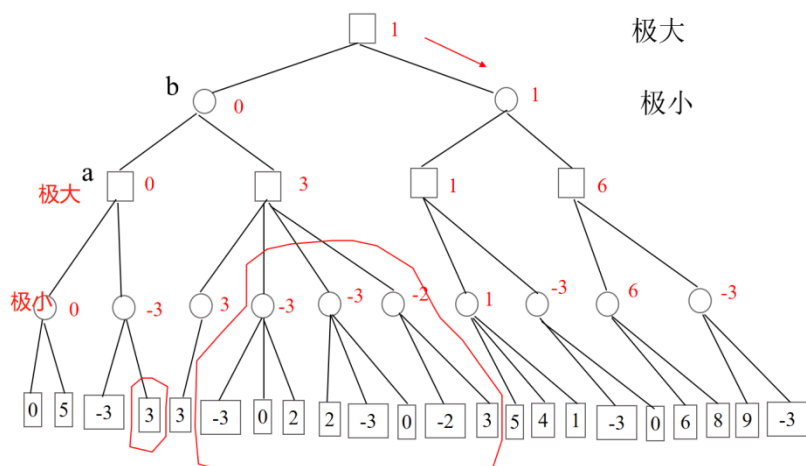
### 极小极大原理

先请冯来切蛋糕，即冯是A，他自然要运用“极小极大原理”。

“极小”指的是B一定会挑选大块，所以留给自己的肯定是小块，也就是表格中的左边一列；

“极大”指的是A要使自己的蛋糕尽量大；

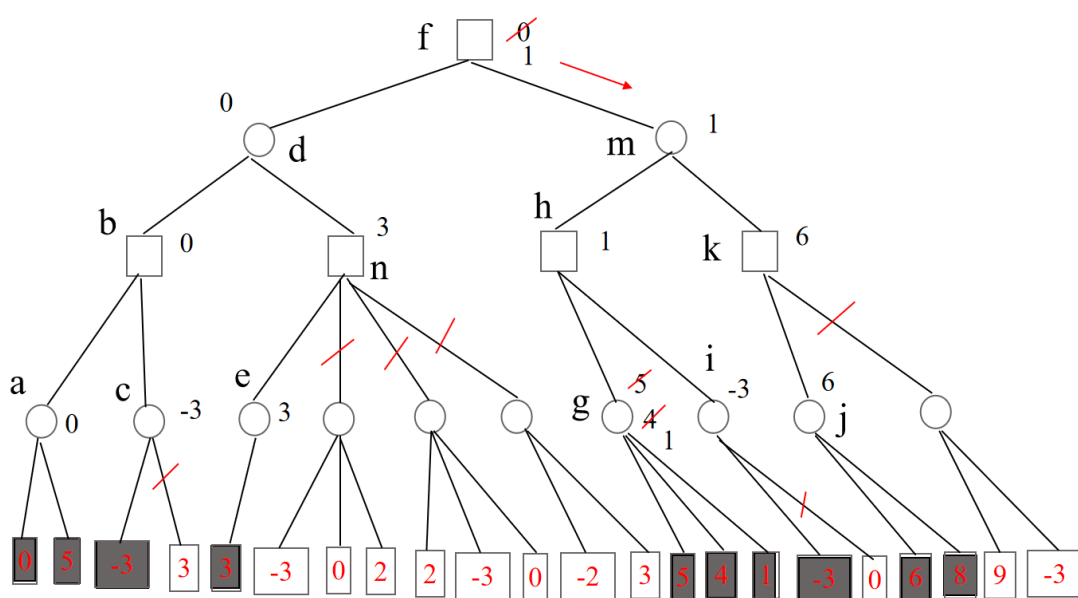
“极小极大”组合起来的意思是，A已知B会选大块，所以会把较小的一块切得大一些，对A来说，最好的结果就是表格的左上角“一半、一半”，即两人各分得半块蛋糕，这就是这个问题的理性解。



$\alpha$  -  $\beta$  剪枝算法:

因为方块代表我方，所以方块肯定取得是两个圆圈中的最大值，为了让对方失败，圆圈肯定要方块的最小值，如果圆圈中已经有个值不被取，那么别圆圈的子方块的值再被圆圈取也只能小于圆圈，但是方块肯定是要取圆圈的最大值，因此，圆圈就不用再去遍历属于自己的子方块了。有此引发剪枝操作。

## $\alpha$ - $\beta$ 剪枝 (续)



剪枝、搜索策略很难改变算法的复杂性

姜桂飞：算法博弈论

主导战略：

- 对于玩家  $i$ ，如果玩  $x$ ，策略  $x$  支配策略  $y$

总比不管其他人怎样都玩  $y$  好

但是你不应该严格的采取这个策略

考虑到回报，她也会选择策略“ $\alpha$ ”。

- 主导战略均衡：（ $\alpha$ ， $\alpha$ ）效率低下，囚徒困境

五个结论：

选择严格优势策略（无论对方选择什么，自己的收益都会更高）

理性选择导致次优结果

Payoffs matter. 苟欲求之，必先知之

策略决策的核心：换位思考，了解别人的举动。

人总是以自己为出发点思考问题。

## 策略：

- 参与人在给定信息集的情况下选择行动的规则，它规定参与人在什么情况下选择什么行动，是参与人的“相机行动方案”。

## 纯策略：

- 如果一个策略规定参与人在每一个给定的信息情况下只选择一种特定的行动，称为纯策略，简称“策略”，即参与人在其策略空间中选取唯一确定的策略。

## 混合策略：

- 如果一个策略规定参与人在给定的信息情况下以某种概率分布随机地选择不同的行动，称为混合策略。参与人采取的不是明确唯一的策略，而是其策略空间上的一种概率分布。

设：政府救济的概率 $\theta=1/2$ ；

不救济的概率 $1-\theta=1/2$ 。

流浪汉寻找工作的期望效用：

$$1/2 \times 2 + 1/2 \times 1 = 1.5$$

流浪的期望效用：

$$1/2 \times 3 + 1/2 \times 0 = 1.5$$

|                | 找工作 $\gamma$  | 游荡 $1-\gamma$ |
|----------------|---------------|---------------|
| 救济 $\theta$    | <u>3</u> , 2  | -1 , <u>3</u> |
| 不救济 $1-\theta$ | -1 , <u>1</u> | <u>0</u> , 0  |

- 此时，流浪汉选择任何混合策略的期望效用都是1.5，则流浪汉的任何一种策略(纯的 $\gamma=1$ 或 $\gamma=0$ 或混合的 $0<\gamma<1$ )都是政府所选择的混合策略的最优反应。

而当

$\gamma=0.2$

$1-\gamma=0.8$

寻找工作 $\gamma$

游荡 $1-\gamma$

游荡

寻找工作

救济 $\theta$

不救济 $1-\theta$

救济:

$3 \times 0.2 - 1 \times 0.8 = -0.2$

不救济:

$-1 \times 0.2 + 0 \times 0.8 = -0.2$

如果流浪汉选择这个混合策略，政府的任何策略(混合的或纯的)带给政府的期望效用为-0.2。特别的，以1/2的概率分别选择救济和不救济当然也是政府对于流浪汉所选择的混合策略的最有反应。

这样，我们得到一个混合策略组合，每一个参与人的混合策略都是给对方混合策略是的最优选择，从而构成混合策略纳什均衡。

|     | 策略         | 期望收益 |
|-----|------------|------|
| 政府  | (0.5, 0.5) | -0.2 |
| 流浪汉 | (0.2, 0.8) | 1.5  |

纳什均衡：

一套策略，每个玩家一个，这样没有一个玩家有动力偏离她的策略，因为其他玩家没有偏离

## Nash Equilibrium

| you \ pair | $\alpha$ | $\beta$  |
|------------|----------|----------|
| $\alpha$   | (0, 0)   | (-1, -3) |
| $\beta$    | (-3, -1) | (1, 1)   |

- $(\alpha, \alpha)$  and  $(\beta, \beta)$  are Nash Equilibria.
- A Dominant-strategy equilibrium is a Nash Equilibrium.

## The Battle of the Sexes

| Girl \ Boy | 夺冠     | 八佰     |
|------------|--------|--------|
|            | 夺冠     | 八佰     |
| 夺冠         | (2, 1) | (0, 0) |
| 八佰         | (0, 0) | (1, 2) |

- Two NEs (夺冠, 夺冠), (八佰, 八佰)
- Which one to choose?

蒙特卡罗树搜索(Monte Carlo Tree Search)并不是一种"模拟人"的算法。而是通过随机的对游戏进行推演来逐渐建立一棵不对称的搜索树的过程。可以看成是某种意义上的强化学习，当然这一点学界还有一些争议。

蒙特卡罗树搜索大概可以被分成四步。选择(Selection)，拓展(Expansion)，模拟(Simulation)，1.Selection（选择）

从根节点开始，根据某种策略依次选择最佳的子节点，直到到达叶子节点。选择节点的好坏直接影响搜索的好坏，目前广泛采用的策略是 [uct 算法](#) (Upper Confidence Bound Apply to Tree)

2. Expansion（扩展）

- 扩展叶子节点，将一个或多个可行的落子添加为该叶子节点的子节点。

3. Simulation（模拟）

- 根据某种策略（比如围棋中的完全随机落子）从扩展的位置进行到游戏结束。

模拟总是会产生一个结果，对于围棋类游戏来说就是获胜、失败或平局，但是广义上来说模拟的合法结果可以是任意值。

4. Backpropagation（反向传播）

- 将模拟的结果沿着传递路径反向传递回根节点。反向传播(Backpropagation)。

### UCT算法 (Upper Confidence Bound applied to Trees)

“Selection is the strategic task that selects one of the children of a given node. It controls the balance between **exploitation** and **exploration**.”

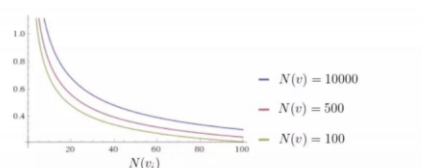
Exploitation(利用): 给定过去的经验选择能期望产生好的回报的动作。

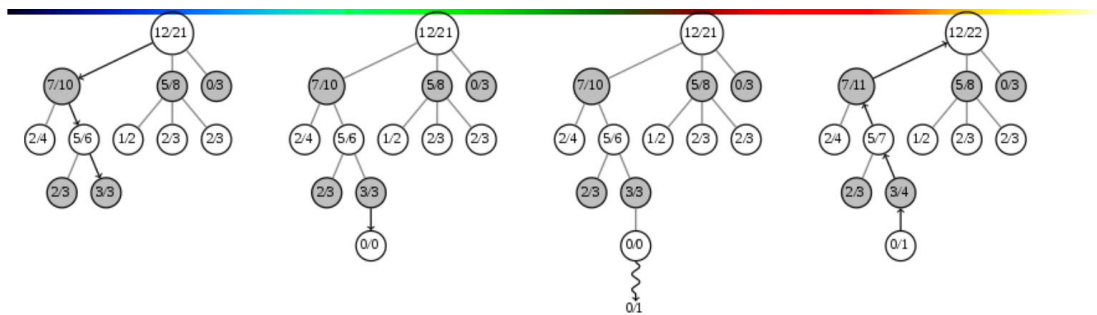
Exploration(探索): 尝试可能能够使得在未来做出更好决策的新事物。

在选择节点的时候，我们目前来讲当然应该考虑收益较高的节点；但同时也要考虑那些由于被探测数量少，暂时收益不高，但在未来很有希望的节点。

$$UCT(v_i, v) = \frac{Q(v_i)}{N(v_i)} + c \sqrt{\frac{\ln(N(v))}{N(v_i)}}$$

公式的第一部分表示截至目前 $v_i$ 节点平均每次的收益  
公式的第二部分则倾向于那些相对较少被探索的节点





- $\text{Score}(7/10) = \text{uct}(7/10, 12/21) = 7/10 + C \sqrt{\frac{\ln(21)}{10}} = 0.7 + 0.55C$
- $\text{Score}(5/8) = \text{uct}(5/8, 12/21) = 5/8 + C \sqrt{\frac{\ln(21)}{5}} = 0.625 + 0.62C$
- $\text{Score}(0/3) = \text{uct}(0/3, 12/21) = 0/3 + C \sqrt{\frac{\ln(21)}{3}} = 0 + 1.00C$

.....

c越大越倾向于广度搜索，也就是探索有潜力的节点；c越小越倾向于深度搜索，也就是多访问在当前已知信息下，平均奖励最高的节点。

机器学习：

使计算机能模拟人的学习行为，自动的通过学习来获取知识和技能

分类：

监督学习：根据教师给出的正确响应

强化学习：给出奖惩，但不给正确答案

信用分配，游戏，迷宫，平衡木

非监督学习：聚类，基因组，管理计算机群，分析社会网，市场划分，天文，信号分离

机械学习：存储足够的信息，通过对知识库的检索得到的知识来求解问题

指导式学习：由外部环境向系统提供指示性建议，转化为知识入库

示例学习：从环境中获取与某概念有关的例子，经归纳得出的一般性概念

迁移学习。做猫狗分类，有少量 label data，另外有其他不属于猫狗大量 data，有的有 label 有的没

## • Supervised learning

- Decision tree induction
- Linear regression
- Logistic regression
- Support vector machines & kernel methods
- Model ensembles
- Bayesian learning
- Neural networks & deep learning
- Learning theory

## • Unsupervised learning

- Clustering
- Dimensionality reduction 降维

## • Reinforcement learning

- Temporal difference learning 时差学习
- Q learning

## • Evaluation

## • Applications

Our focus will be on applying machine learning to real applications

机器学习=找函数

机器学习框架分为 training 和 testing。

学习部分：准备一个 model (a set of function)，准备训练数据 (training data)，经过学习，找出最好的 function。

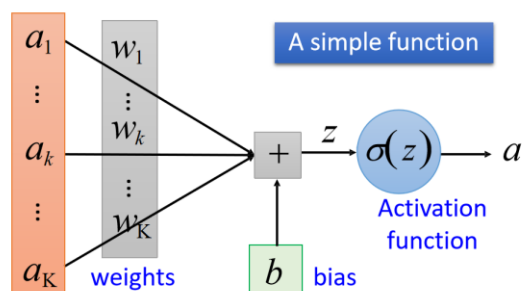
测试部分：找未曾训练过的数据，输入后，能得到正确输出。

机器学习的三个步骤：定义一个 model，能够判断方法好或者不好，能够从 model 中挑选出最好的方法。

神经网络：

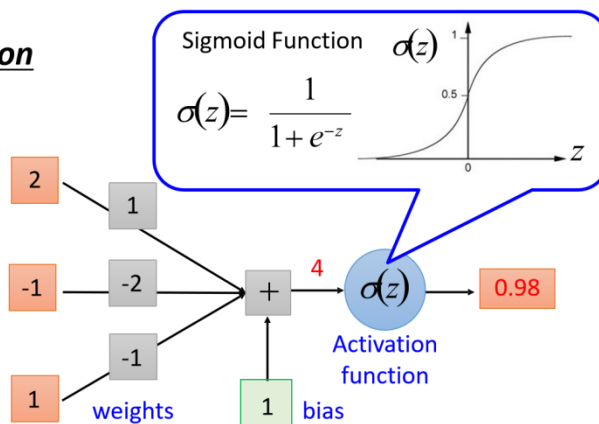
### Neuron

$$z = a_1 w_1 + \dots + a_k w_k + \dots + a_K w_K + b$$



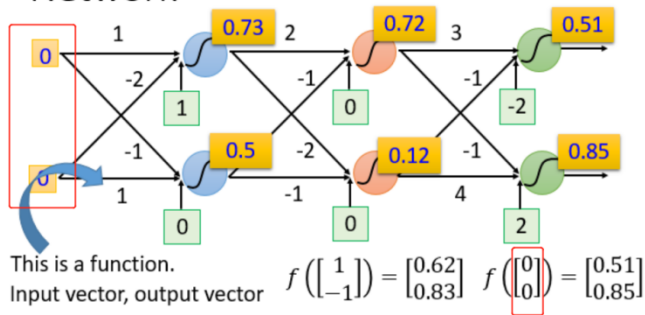
激活函数是 sigmoid

### Neuron



每个神经元都有自己的权重和偏差。

## Fully Connect Feedforward Network



Given network structure, define **a function set**

给定网络结构，相当于定义了一个函数集，每个神经元可以使用不同的函数，那么整个网络的函数集合是非常大，所以说深度学习可以拟合任意曲线。

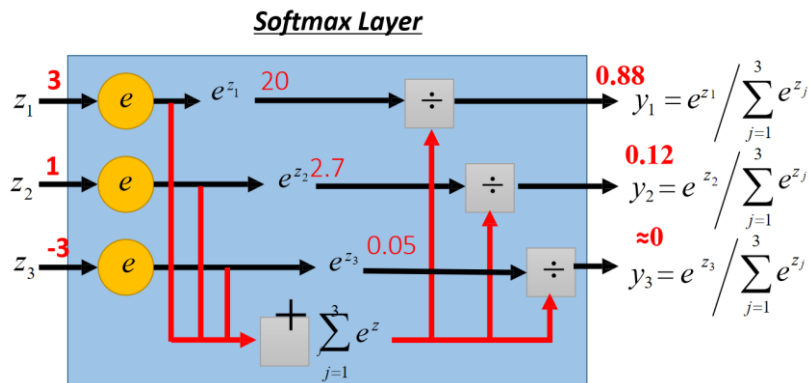
全链接：layer1 与 layer2 之间两两都有连接

前馈：传递的方向是由后往前传

输入层不是有神经元组成的，严格讲不能当作一个 layer

除了输入层和输出层，其他的叫做隐藏层

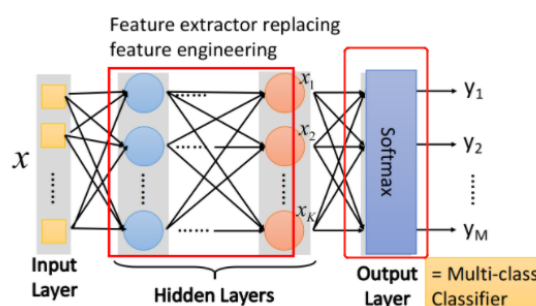
- Softmax layer as the output layer



## Output Layer

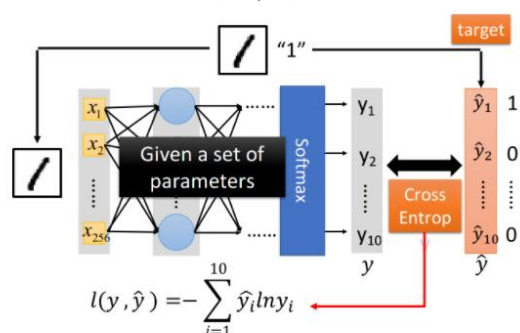
- hidden layers这看做是一个\*\*feature extractor(特征提取器)\*\*经过这个feature extractor得到的 $x_1, x_2, \dots, x_k$ 就可以被当作一组新的feature
- output layer: 一个**Multi-class classifier**, 它是拿经过feature extractor转换后的那一组比较好的feature(能够被很好地separate)进行分类的, 由于我们把output layer看做是一个Multi-class classifier, 所以我们会在最后一个layer加上**softmax**

### Output Layer as Multi-Class Classifier



定义一个function的好坏, 由于现在我们做的是个Multi-class classification, 所以image为数字1的label "1"告诉我们, 现在的target是一个10维的vector, 只有在第一维对应数字1的地方, 它的值是1, 其他都是0

### Loss for an Example

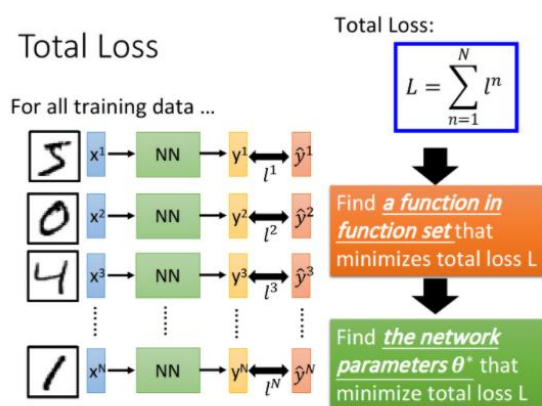


- input这张image的256个pixel, 通过这个neural network之后, 会得到一个output, 称之为 $y$ ; 而从这张image的label中转化而来的target, 称之为 $\hat{y}$ , 有了output  $y$ 和target  $\hat{y}$ 之后, 要做的事情是计算它们之间的cross entropy(交叉熵), 这个做法跟我们之前做Multi-class classification的时候是一模一样的

$$\text{Cross Entropy} : l(y, \hat{y}) = - \sum_{i=1}^{10} \hat{y}_i \ln y_i$$

### Step 3: Pick the best function

- 接下来就去调整参数，让这个cross entropy越小越好，当然整个training data里面不会只有一笔data，你需要把所有data的cross entropy都sum起来，得到一个total loss  $L = \sum_{n=1}^N l^n$ ，得到loss function之后你要做的事情是找一组network的parameters:  $\theta^*$ ，它可以minimize这个total loss，这组parameter对应的function就是我们最终训练好的model

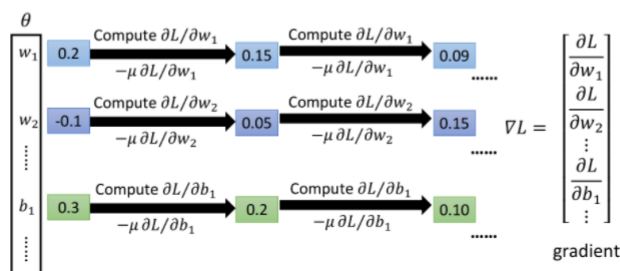


那怎么去找这个使total loss minimize的 $\theta^*$ 呢？使用的方法就是我们的老朋友——**Gradient Descent**

那怎么去找这个使total loss minimize的 $\theta^*$ 呢？使用的方法就是我们的老朋友——**Gradient Descent**

- 现在你的 $\theta$ 里面是一大堆的weight、bias参数，先random找一个初始值，接下来去计算每一个参数对total loss的偏微分，把这些偏微分全部集合起来，就叫做gradient，有了这些偏微分以后，你就可以更新所有的参数，都减掉learning rate乘上偏微分的值，这个process反复进行下去，最终找到一组好的参数，就做完了deep learning的training了

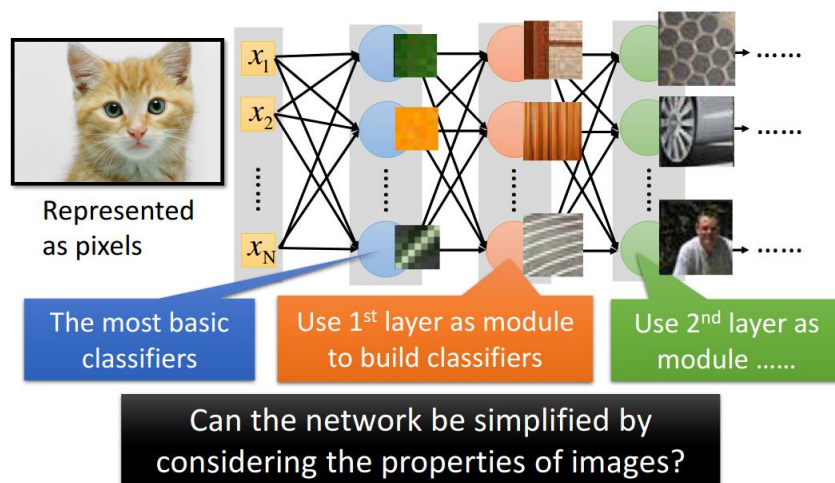
### Gradient Descent



用梯度下降法，不断更新  $w$  的值，用  $w - \mu \partial L / \partial w$

# Why CNN for Image?

[Zeiler, M. D., ECCV 2014]



第一个 layer 的 neuron，它是最简单的 classifier，它做的事情就是 detect 有没有绿色出现、有没有黄色出现、有没有斜的条纹出现等等

那第二个 layer，它做的事情是 detect 更复杂的东西，根据第一个 layer 的 output，它如果看到直线横线，就是窗框的一部分；如果看到棕色的直条纹就是木纹；看到斜条纹加灰色的，这个有可能是很多东西，比如说，轮胎的一部分等等

再根据第二个 hidden layer 的 output，第三个 hidden layer 会做更复杂的事情，比如它可以知道说，当某一个 neuron 看到蜂巢，它就会被 activate；当某一个 neuron 看到车子，它就会被 activate；当某一个 neuron 看到人的上半身，它就会被 activate 等等

那现在的问题是样子：当我们直接用一般的 fully connected 的 feedforward network 来做图像处理的时候，往往会需要太多的参数

举例来说，假设这是一张  $100 \times 100$  的彩色图片，它的分辨率才  $100 \times 100$ ，那这已经是很小张的 image 了，然后你需要把它拉成一个 vector，总共有  $100 \times 100 \times 3$  个 pixel，把这些加起来 input vector 就已经有三万维了；如果 input vector 是三万维，又假设 hidden layer 有 1000 个 neuron，那仅仅是第一层 hidden layer 的参数就已经有  $30000 \times 1000$  个了，这样就太多了

所以，CNN：来简化这个 neural network 的架构，我们根据自己的知识和对图像的理解，一开始就把某些实际上用不到的参数给过滤掉，不用 fully connected network，而是用比较少的参数 CNN 其实是比一般的 DNN 还要更简单的。

我们可以对一张 image 做 subsampling(二次抽样)，假如你把它奇数行、偶数列的 pixel 拿掉，image 就可以变成原来的十分之一大小，而且并不会影响人对这张 image 的理解

一定要看下面的链接：

卷积-最大池-卷积-最大池

<https://blog.csdn.net/chairon/article/details/110088441>

下面的链接是 RNN（循环神经网络）的

<https://blog.csdn.net/chairon/article/details/112093821>

词嵌入模型：

简单来说，embedding 就是用一个低维的向量表示一个物体，可以是一个词，或是一个商品，或是一个电影等等。这个 embedding 向量的性质是能使距离相近的向量对应的物体有相近的含义，比如 Embedding(复仇者联盟)和 Embedding(钢铁侠)之间的距离就会很接近，但 Embedding(复仇者联盟)和 Embedding(乱世佳人)的距离就会远一些